

Any-Order Adaptive Editing for Masked Diffusion LLMs

Abstract

Masked diffusion language models (MDLMs) generate text by iteratively unmasking tokens in parallel, but current inference relies on static confidence thresholds and fixed block schedules that leave the paradigm’s any-order planning potential largely untapped. We propose **Any-Order Adaptive Editing (AOAE)**, a speculative dual-model framework that exploits the natural dual-speed structure of Mixture-of-Experts (MoE) MDLMs. A fast *hard-routed auxiliary* ($\sim 1.4\text{B}$ active parameters) drafts token predictions whose KV states are pre-cached, while a slow *soft-routed primary* (all 16B parameters) verifies and refines—reusing cached states where the two agree and recomputing only where they disagree. A lightweight steering policy jointly decides where to unmask, where to remask for self-correction, and which positions to commit to cache, trained end-to-end via GRPO. We further introduce *composed prediction* to bias token selection toward cache-aligned choices, and frame diffusion caching as an access-pattern prediction problem rather than an access-pattern regularization problem.

1 Introduction

Masked diffusion language models (MDLMs) generate text by reversing a token-masking process, producing all positions in parallel rather than left-to-right [1, 2, 3, 4]. This enables *any-order* generation: the model can resolve dependencies in whatever sequence best suits the problem. Recent scaling has produced frontier-class MDLMs—LLaDA 2.0 at 100B parameters [6] and LLaDA 2.1 with Token-to-Token (T2T) editing for retroactive error correction [7]—but inference remains governed by static heuristics. LLaDA 2.1 uses fixed confidence thresholds τ_{mask} and τ_{edit} and decodes in rigid semi-autoregressive blocks; Fast-dLLM [8] applies a single confidence threshold for variable-rate unmasking. These approaches require per-task tuning, degrade on long sequences [9], and impose uniform compute regardless of position difficulty.

Three lines of work have begun to address these limitations. Kim et al. [10] showed that adaptive token ordering boosts MDM accuracy on Sudoku from $<7\%$ to $\approx 90\%$, and Jazbec et al. [9] trained a lightweight RL policy to decide *which* tokens to unmask, outperforming heuristic samplers—but their formulation covers only unmasking, ignoring self-correction and caching. PRISM [11] provides model-agnostic self-correction via remasking, but uses a fixed threshold rather than a learned policy. dKV-Cache [12] and Elastic-Cache [13] accelerate inference by caching stable KV states, but they *constrain* the decoding trajectory to suit the cache (block-based or prefix-restricted unmasking), sacrificing diffusion’s any-order flexibility.

We propose **Any-Order Adaptive Editing (AOAE)** to unify all three—adaptive steering, self-correction, and speculative caching—in a single framework built on a novel observation: MoE models naturally provide a dual-speed architecture. With standard hard top- k routing, LLaDA 2.1-mini activates $\sim 1.4\text{B}$ of 16B parameters per token (fast); with temperature-controlled soft routing, all 16B parameters participate (slow but more expressive). AOAE pairs these as a **hard-routed auxiliary** and a **soft-routed primary** in a speculative diffusion loop analogous to speculative decoding in autoregressive LLMs [16, 17]—but fundamentally richer, since all positions are simultaneously active and the primary can accept, reject, or refine any subset in parallel.

2 Background and Notation

2.1 Masked Diffusion Models

Let $\mathbf{x}_0 = (x_0^1, \dots, x_0^L) \in \mathcal{V}^L$ be a token sequence with vocabulary $\mathcal{V}$ and mask token $[\mathbf{M}]$. The forward process independently corrupts each token:

$$q(x_t^k | x_0^k) = \begin{cases} 1 - t, & \text{if } x_t^k = x_0^k, \\ t, & \text{if } x_t^k = [\mathbf{M}], \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

for $t \in [0, 1]$, so at $t=1$ all tokens are masked. A denoising network p_θ is trained via the ELBO [5, 2]:

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} \left[\frac{1}{t} \sum_{k=1}^L \mathbb{I}[x_t^k = [\mathbf{M}]] \log p_\theta(x_0^k | \mathbf{x}_t) \right]. \quad (2)$$

At inference, the model starts from $\mathbf{y}_T = ([\mathbf{M}], \dots, [\mathbf{M}])$ and at each step t computes per-position distributions $p_t^k := p_\theta(y_0^k = \cdot | \mathbf{y}_t)$. A sampling strategy selects a subset $\mathcal{U}_t \subseteq \mathcal{M}_t$ of the masked positions $\mathcal{M}_t := \{k : y_t^k = [\mathbf{M}]\}$ to unmask:

$$y_{t-1}^k = \begin{cases} y \sim p_t^k, & \text{if } k \in \mathcal{U}_t, \\ y_t^k, & \text{otherwise.} \end{cases} \quad (3)$$

The choice of $\mathcal{U}_t$ critically affects both speed and quality [9, 8].

2.2 LLaDA 2.1: Draft-and-Edit Decoding

LLaDA 2.1 [7] extends unmasking with T2T editing via confidence thresholds:

$$\Gamma_t = \{k \in \mathcal{M}_t \mid \max_v p_\theta(v | \mathbf{y}_t) > \tau_{\text{mask}}\} \quad (\text{M2T unmasking}), \quad (4)$$

$$\Delta_t = \{k \notin \mathcal{M}_t \mid \max_v p_\theta(v | \mathbf{y}_t) > \tau_{\text{edit}} \wedge \arg \max_v \neq y_t^k\} \quad (\text{T2T editing}). \quad (5)$$

Two modes are offered: *Speedy* (S) with aggressive thresholds and *Quality* (Q) with conservative ones, but both are fixed hyperparameters.

2.3 KV-Caching in Diffusion Models

In MDLMs, bidirectional attention causes KV states to evolve across steps, making naive caching fragile. dKV-Cache [12] caches high-confidence positions and recomputes only changed ones, achieving 2–10 $\times$ speedup. The per-token, per-layer *KV drift* quantifies staleness:

$$\Delta_{t,\ell}^k := \left\| K_{t,\ell}^k - K_{t-1,\ell}^k \right\|_2 + \left\| V_{t,\ell}^k - V_{t-1,\ell}^k \right\|_2. \quad (6)$$

Drift is empirically small for most steps and grows with layer depth, suggesting that recomputing shallow layers at every step is wasteful.

Elastic-Cache [13] exploits this structure with three mechanisms: (i) locality via a sliding window $\mathcal{M}_t^\beta$ over masked positions; (ii) an attention-derived staleness proxy based on cosine similarity of attention weights to the most-attended decoded token:

$$\sigma_{t,\ell} := \frac{\langle S_{t-1,\ell}[\mathcal{M}_t^\beta, T_{t-1,\ell}], S_{t,\ell}[\mathcal{M}_t^\beta, T_{t-1,\ell}] \rangle}{\|S_{t-1,\ell}[\mathcal{M}_t^\beta, T_{t-1,\ell}]\|_2 \|S_{t,\ell}[\mathcal{M}_t^\beta, T_{t-1,\ell}]\|_2}, \quad (7)$$

where $T_{t,\ell} := \arg \max_{k \in \mathcal{D}_{<t}} \sum_{q \in \mathcal{M}_t^\beta} S_{t,\ell}[q, k]$; and (iii) depth-selective refresh from a boundary layer $\ell_t^* := \min\{\ell : \sigma_{t,\ell} < \gamma\}$, recomputing only layers $\ell > \ell_t^*$.

These methods improve cache efficiency but do so by *constraining* the decoding trajectory—block-based scheduling, prefix-restricted unmasking—trading diffusion’s any-order flexibility for a more regular access pattern.

2.4 Self-Correction via PRISM

PRISM [11] trains a lightweight adapter to estimate per-token quality scores q_t^k —the probability that a generated token matches the ground truth—via binary cross-entropy on (corrupted, clean) pairs. Tokens with q_t^k below a threshold are *remasked* (reverted to [M]) and regenerated. This is strictly simpler than T2T editing: remasking defers to the base model’s well-trained M2T denoising rather than learning a replacement distribution conditioned on the current (possibly incorrect) token, preserving the any-order property.

2.5 Learned Unmasking Policies

Jazbec et al. [9] formalize MDLM sampling as an MDP where a single-layer transformer policy π_ϕ outputs per-position Bernoulli unmasking decisions $u_t^k \sim \text{Ber}(\sigma(f_\phi(c_t^k)))$ based on confidences $c_t^k := \max_v p_t^k(v)$, trained via GRPO [15] with a multiplicative reward:

$$R(\mathbf{y}, \mathbf{y}_t) = r(\mathbf{y}, \mathbf{y}_t) \cdot \left(1 - \frac{T-t}{T}\right)^\alpha, \quad (8)$$

where $r(\cdot)$ is task-specific correctness and α controls the speed–accuracy tradeoff. This outperforms heuristic samplers but addresses only unmasking—no self-correction, caching, or speculative inference.

3 Method: Any-Order Adaptive Editing

AOAE extends the learned-policy framework of Section 2.5 to a full speculative diffusion architecture, formalized as a finite-horizon MDP $(\mathcal{S}, \mathcal{A}, P, R, T)$ over a dual-model system.

3.1 Dual-Model MoE Architecture

LLaDA 2.1-mini is a 16B-parameter MoE model with $E=128$ routed experts per layer, of which $k=8$ are selected per token via hard top- k routing, so only $\sim 1.4\text{B}$ parameters are active per forward pass. AOAE instantiates two copies of the same frozen model with different routing:

Hard-routed auxiliary (fast). Standard hard top- k routing ($\sim 1.4\text{B}$ active). Serves as the draft model: produces token predictions for all positions, whose KV states are pre-cached.

Soft-routed primary (slow, expressive). Replaces hard routing with temperature-controlled soft routing:

$$w_i^{\text{soft}} = \frac{\exp(z_i/\tau_r)}{\sum_{j=1}^E \exp(z_j/\tau_r)}, \quad i = 1, \dots, E, \quad (9)$$

where z_i are router logits and $\tau_r > 0$ is the routing temperature. At $\tau_r \rightarrow 0$ this recovers hard routing; at $\tau_r > 0$ all experts participate. Both copies share frozen weights; only the routing function

differs. The dual setup requires $2\times$ model memory but not $2\times$ compute, since the auxiliary’s KV states are reused at agreement positions.

Routing temperature is the single knob controlling quality vs. throughput. At $\tau_r \rightarrow 0$, the primary matches the auxiliary: near-perfect agreement ($\bar{\alpha}_t \approx 1$), maximal cache reuse, minimal quality gain. At $\tau_r = 0.01$, $>99\%$ of routing weight remains on the top- k experts, yielding high agreement with modest quality improvement. At $\tau_r = 0.1$, meaningful divergence: richer predictions but agreement drops to $\sim 70\text{--}80\%$. At $\tau_r = 0.5$, all experts contribute substantially; quality may improve but the policy must be highly selective about caching.

3.2 State: Soft-Masked Representations

The policy’s state must convey predictive uncertainty, not just binary mask indicators. Following Hersche et al. [14], we construct a soft-masked embedding per position. Let $p_t^k \in \Delta^{|\mathcal{V}|-1}$ be the primary’s predictive distribution and $c_t^k := \max_v p_t^k(v)$ the confidence. The soft-masked embedding is:

$$\mathbf{h}_t^k = \lambda(p_t^k) \mathbf{E}_{[\mathbf{M}]} + (1 - \lambda(p_t^k)) \sum_{j \in \text{top-}K(p_t^k)} \pi_j^k \mathbf{E}_{v_j}, \quad (10)$$

where $\pi_j^k = p_t^k(v_j) / \sum_{j'} p_t^k(v_{j'})$ are renormalized top- K probabilities and the gating function is:

$$\lambda(p_t^k) = \omega_s \cdot \sigma(\omega_a(-H(p_t^k) - \omega_b)), \quad (11)$$

with entropy $H(p_t^k)$ and learnable parameters $\omega = (\omega_s, \omega_a, \omega_b)$. At low confidence (high entropy), $\lambda \rightarrow \omega_s$ and the mask embedding dominates; at high confidence, $\lambda \rightarrow 0$ and the top- K token embeddings dominate.

The full state is $s_t = (\mathbf{H}_t, \mathbf{m}_t, \mathbf{q}_t, \alpha_t, t/T)$, where $\mathbf{H}_t = (\mathbf{h}_t^1, \dots, \mathbf{h}_t^L)$, $m_t^k = \mathbb{I}[y_t^k = [\mathbf{M}]]$, q_t^k is the PRISM quality score, and $\alpha_t^k = \mathbb{I}[\arg \max_v d_t^k(v) = \arg \max_v p_t^k(v)]$ is the auxiliary–primary agreement signal.

3.3 Action: Unified Unmask–Remask–Cache Decisions

The policy outputs three per-position Bernoulli vectors:

$$u_t^k \sim \text{Ber}(\sigma(f_{\phi^{\text{unmask}}}(s_t)^k)), \quad (\text{Unmask}), \quad (12)$$

$$r_t^k \sim \text{Ber}(\sigma(f_{\phi^{\text{remask}}}(s_t)^k)), \quad (\text{Remask}), \quad (13)$$

$$\kappa_t^k \sim \text{Ber}(\sigma(f_{\phi^{\text{cache}}}(s_t)^k)), \quad (\text{Cache}). \quad (14)$$

The full action $\mathbf{a}_t = (\mathbf{u}_t, \mathbf{r}_t, \kappa_t) \in \{0, 1\}^{3L}$ has a tractable factorized likelihood:

$$\pi_{\phi}(\mathbf{a}_t | s_t) = \prod_{k=1}^L \prod_{d \in \{\text{u}, \text{r}, \text{c}\}} (\sigma(f_{\phi^d}(s_t)^k))^{a_t^{d,k}} (1 - \sigma(f_{\phi^d}(s_t)^k))^{1-a_t^{d,k}}. \quad (15)$$

Validity is enforced via logit masking before the sigmoid: unmask applies only to masked positions ($u_t^k = 0$ if $m_t^k = 0$); remask applies only to unmasked positions ($r_t^k = 0$ if $m_t^k = 1$); cache and remask are mutually exclusive ($\kappa_t^k \cdot r_t^k = 0$).

We choose unmasking and remasking over the T2T editing approach since the latter requires the base model to produce a replacement token conditioned on the current (possibly incorrect) token—learning $p(x_0^k | \mathbf{x}_t, y_t^k \neq [\mathbf{M}])$ on artificially corrupted data. This is harder than the standard M2T objective $p(x_0^k | \mathbf{x}_t, y_t^k = [\mathbf{M}])$ and breaks the any-order property. Remasking simply returns a position to $[\mathbf{M}]$, after which the base model’s well-trained denoising handles it.

Algorithm 1 AOA Speculative Diffusion Inference

```
1: Input: Auxiliary  $p_{\theta}^{\text{hard}}$ , primary  $p_{\theta}^{\text{soft}}$ , PRISM  $q_{\psi}$ , policy  $\pi_{\phi}$ , prompt  $\mathbf{x}$ , steps  $T$ 
2: Init:  $\mathbf{y}_T \leftarrow ([M], \dots, [M])$ ,  $\mathcal{K} \leftarrow \emptyset$ 
3: for  $t = T, T-1, \dots, 1$  do
4:   Phase 0 (Draft):  $\{d_t^k\} \leftarrow p_{\theta}^{\text{hard}}(\cdot \mid \mathbf{x}, \mathbf{y}_t)$ ; pre-cache KV states  $\triangleright \sim 1.4\text{B}$  active
5:   Phase 0b (Verify):  $\{p_t^k\} \leftarrow p_{\theta}^{\text{soft}}(\cdot \mid \mathbf{x}, \mathbf{y}_t; \mathcal{K})$ ;  $\{q_t^k\} \leftarrow q_{\psi}(\mathbf{y}_t)$   $\triangleright$  all 16B active
6:   Compute  $\alpha_t^k = \mathbb{I}[\arg \max d_t^k = \arg \max p_t^k]$ ; construct  $s_t$  via Equation (10)
7:    $(\mathbf{u}_t, \mathbf{r}_t, \boldsymbol{\kappa}_t) \sim \pi_{\phi}(\cdot \mid s_t)$ 
8:   for  $k : r_t^k=1, m_t^k=0$  do  $\triangleright$  Phase 1: Remask
9:      $y_{t-1}^k \leftarrow [M]$ ;  $\mathcal{K} \leftarrow \mathcal{K} \setminus \{k\}$ 
10:  end for
11:   $\tilde{p}_t^k \leftarrow \text{COMPOSE}(p_t^k, d_t^k, \alpha_t^k, \gamma)$   $\triangleright$  Equation (16)
12:  for  $k : u_t^k=1, m_t^k=1$  do  $\triangleright$  Phase 2: Unmask
13:     $y_{t-1}^k \sim \tilde{p}_t^k$ 
14:  end for
15:  for  $k : \kappa_t^k=1, \alpha_t^k=1, r_t^k=0$  do  $\triangleright$  Phase 3: Cache
16:     $\mathcal{K} \leftarrow \mathcal{K} \cup \{k\}$ 
17:  end for
18: end for
19: Return  $\mathbf{y}_1$ 
```

3.4 Inference: Speculative Dual-Model Loop

At each step (Algorithm 1), the auxiliary drafts predictions and pre-caches KV states; the primary verifies, computing agreement α_t^k ; the policy outputs joint actions; remasks are applied with cache invalidation; unmasks sample from a composed distribution; and cache commits are made at agreement positions.

Following [9], if $\mathbf{u}_t = \mathbf{0}$ and $\mathcal{M}_t \neq \emptyset$, the highest-confidence masked position is unmasked. This fallback is disabled during training to prevent reward hacking.

3.5 Composed Prediction

The auxiliary and primary share weights and differ only in routing, so at low τ_r their predictions are highly correlated. We compose the auxiliary’s draft distribution d_t^k with the primary’s p_t^k :

$$\tilde{p}_t^k(v) \propto p_t^k(v) \cdot d_t^k(v)^{\gamma \cdot \alpha_t^k}, \quad (16)$$

where $\gamma \geq 0$ controls composition strength. When the models agree ($\alpha_t^k = 1$), the composed distribution concentrates mass on the shared high-probability token, increasing cache reuse. When they disagree ($\alpha_t^k = 0$), it reduces to the primary alone. Since composition sharpens only at agreement positions, overall entropy reduction is small and the any-order property is preserved where it matters most.

During GRPO training, composed prediction is active, so the policy learns to account for it when predicting cache actions. The cache-thrashing penalty naturally teaches the policy to cache only at high-agreement positions.

3.6 Policy Architecture

The steering policy π_{ϕ} is a lightweight transformer:

1. **Input:** Per-position features $(\mathbf{h}_t^k, m_t^k, q_t^k, \alpha_t^k, t/T)$ projected to dimension d_π .
2. **Backbone:** N_π -layer transformer with d_π -dimensional hidden states and full bidirectional attention.
3. **Heads:** Three linear projections producing $f_{\phi^{\text{unmask}}}, f_{\phi^{\text{remask}}}, f_{\phi^{\text{cache}}}$ logits per position.

With $N_\pi = 1$ and $d_\pi = 128$, this is $\sim 500\text{K}$ parameters ($< 0.01\%$ of the base model). The composed prediction mechanism (Equation (16)) is the standard technique for producing the token distributions from which unmask, remask, and cache decisions are derived: the agreement signal α_t^k biases the composed distribution toward the auxiliary’s draft at agreement positions, directly informing the policy’s cache logits, while PRISM quality scores q_t^k drive remask decisions.

3.7 Positional Speculative Caching

The speculative loop in Section 3.4 focuses on *token-level* speculation: a fast auxiliary drafts token values whose KV states are pre-cached, and a slower primary verifies and reuses caches when the two agree. In addition, we study a complementary *position-level* approach that speculates *where* computation will be needed next, rather than *what* token will be placed. This *next- H positional speculation* is motivated by the observation that diffusion inference induces a nontrivial, instance-dependent *access pattern* over positions (which tokens are revisited, edited, or refreshed next), especially under remasking. Unlike the dual-model draft/verify strategy, positional speculation does *not* require the underlying MDLM to have an MoE architecture: it can be applied to any diffusion LLM that exposes token-wise uncertainty signals and supports KV reuse.

Refresh / prefetch set. Let $\mathcal{I} := \{1, \dots, L\}$ denote all response positions. At each denoising step t , we define an *access (refresh) set* $\mathcal{Q}_t \subseteq \mathcal{I}$ that determines which positions receive compute-intensive cache refresh (e.g., QKV recomputation and/or KV prewarming). Positions that the policy actually modifies must be included:

$$\mathcal{Q}_t \supseteq \{k : u_t^k = 1\} \cup \{k : r_t^k = 1\}. \quad (17)$$

Beyond these mandatory updates, we allow a budgeted number of additional positions to be refreshed *speculatively*.

Next- H access prediction under a refresh budget. We introduce a lightweight *access head* $g_\phi^{(H)} : \mathcal{S} \rightarrow \mathbb{R}^L$ that outputs per-position logits predicting whether each position will be accessed within the next H steps. Concretely, we define a top- B speculative candidate set:

$$\mathcal{C}_t^{(H)} := \text{Top-}B(\{g_\phi^{(H)}(s_t)^k\}_{k=1}^L), \quad \mathcal{Q}_t := \mathcal{C}_t^{(H)} \cup \{k : u_t^k = 1 \vee r_t^k = 1\}, \quad (18)$$

where B is a *refresh budget* controlling the maximum number of positions we keep “warm” per step (in addition to mandatory modifications).

In contrast to token-level speculative drafting, which proposes *token values*, our positional speculative caching predicts *where* computation will be needed next by forming a budgeted access set $\mathcal{Q}_t$ (via next- H candidate selection). This is closely related to training-free diffusion-cache policies that exploit locality and temporal coherence of KV states by refreshing only a small active region while reusing the remainder. When supported by the serving stack, the same candidate set can also be used to *prefetch* KV/activations for likely-soon positions (e.g., via a cheap auxiliary pass or shallow-layer precompute), but this is an implementation optimization rather than a modeling

assumption and does not alter the diffusion trajectory. Crucially, $\mathcal{Q}_t$ is *predicted* from the model state s_t rather than *prescribed* by a fixed schedule (e.g., blockwise or left-to-right windows). In this sense, we treat diffusion caching as an *access-pattern prediction* problem: the policy learns which positions are likely to be accessed soon and allocates refresh/prefetch budget accordingly, without constraining the underlying unmask/remask trajectory to follow a predetermined pattern. The access head leverages two empirically observed regularities in diffusion decoding: *temporal coherence* (KV/attention states typically change slowly across adjacent denoising steps except during bursty revisions) and *locality* (a relatively small subset of positions tends to dominate the next few updates). Rather than enforcing a fixed block/window schedule, we may use these regularities as *predictive signals* in s_t (e.g., uncertainty/quality, recency of previously accessed tokens via age a_t^k , and optionally attention-derived features) to infer which positions are likely to be accessed within the next H steps and should therefore be refreshed/prefetched. Concretely, we refresh a position only when its predicted access probability over the next H steps is high *and* its staleness/quality score exceeds a threshold, while positions with high predicted access but low staleness are merely prefetched.

Layer-wise refresh depth. Empirical analyses of diffusion KV dynamics indicate that drift typically increases with layer depth, suggesting that recomputing shallow layers at every step is often redundant. Motivated by depth-selective refresh policies in drift-aware caching, we optionally predict a per-step boundary layer ℓ_t^* that controls where recomputation begins for positions in $\mathcal{Q}_t$:

$$\ell_t^* \sim \text{Cat}(\text{softmax}(h_\phi(s_t))), \quad \text{recompute layers } \ell > \ell_t^* \text{ for } k \in \mathcal{Q}_t, \quad (19)$$

while reusing cached states for layers $\ell \leq \ell_t^*$. We learn ℓ_t^* within the same GRPO loop (3.8) as the other policy decisions by treating it as an additional stochastic action: boundaries that are too shallow increase cache staleness (leading to more disagreement, remasking, and thrashing), while boundaries that are too deep reduce thrashing but pay more compute. For stability, we optionally warm-start this head by imitating an attention-triggered boundary (Elastic-Cache style) on training trajectories, and then fine-tune with GRPO. We compare learned ℓ_t^* to an attention-triggered baseline and to fixed depth schedules in Section 4.

3.8 Training via GRPO

The base model and PRISM adapter remain frozen; only the policy and soft-mask gating parameters are trained via GRPO [15].

Reward.

$$R(\mathbf{y}^*, \mathbf{y}_{\hat{T}}, \hat{T}) = r(\mathbf{y}^*, \mathbf{y}_{\hat{T}}) \cdot \left(\frac{\hat{T}}{T}\right)^\alpha - \beta \sum_{t=1}^T \text{Thrash}(t), \quad (20)$$

where $r(\cdot)$ is task correctness (binary for math), $\hat{T} = \max\{t : \mathcal{M}_t \neq \emptyset\}$ is the completion step, $\alpha \geq 0$ controls the speed penalty, and $\text{Thrash}(t) = \sum_k \mathbb{I}[k \in \mathcal{K}_{t-1} \wedge r_t^k = 1]$ counts cache invalidations from remasking, penalized by $\beta \geq 0$. The multiplicative structure ensures incorrect generations receive zero reward regardless of speed, preventing the reward hacking observed with additive formulations [9].

GRPO objective. For each prompt, G trajectories are sampled under $\pi_{\phi_{\text{old}}}$ with greedy base-model decoding, so variation arises solely from the policy’s stochastic actions. With advantage

$A^g = R^g - \bar{R}$ and importance ratio $\rho_t^g = \pi_\phi(\mathbf{a}_t^g \mid s_t^g) / \pi_{\phi_{\text{old}}}(\mathbf{a}_t^g \mid s_t^g)$:

$$\mathcal{J}(\phi) = \mathbb{E}_{\mathbf{x}, \mathbf{y}^*} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{T - \hat{T}_g} \sum_{t=\hat{T}_g}^T \min \left\{ \rho_t^g A^g, \text{clip}(\rho_t^g, 1-\epsilon, 1+\epsilon) A^g \right\} \right]. \quad (21)$$

KL regularization is omitted since the policy is trained from scratch.

3.9 Implicit Behavioral Distillation

An important secondary effect of the AOA training loop is that it performs a form of behavioral distillation from the soft-routed primary to the hard-routed auxiliary, without an explicit distillation loss.

The GRPO reward penalizes incorrect outputs and cache thrashing while rewarding speed. To maximize this reward, the policy must learn to: (i) trust the auxiliary’s draft at positions where it agrees with the primary (cache these for speed); (ii) override the auxiliary at positions where it disagrees (defer to the primary’s richer predictions); and (iii) remask positions where neither model is confident (invoke PRISM-guided self-correction). Over training, the policy effectively learns a *map* from the auxiliary’s prediction quality to the appropriate action, and the composed prediction mechanism (Equation (16)) further biases the system toward the auxiliary’s drafts where they are reliable.

The net result is that the hard-routed auxiliary—originally a fast but less expressive model—acquires, through the policy’s mediation, behaviors it could not learn on its own: PRISM-style self-correction (via the remask head), cache-aware generation (via the cache head), and adaptive unmasking (via the unmask head). This is analogous to knowledge distillation [18] in that a smaller/faster model’s effective behavior is improved by a larger/slower model, but the transfer happens through RL-optimized action selection rather than through a KL loss on output distributions. The distillation is “free” in the sense that it is a byproduct of the same GRPO objective that optimizes inference efficiency.

4 Experimental Design

Setup. LLaDA 2.1-mini (16B MoE, frozen) in dual-model configuration. Policy: 1-layer transformer ($\sim 500\text{K}$ parameters, input dimension $D+4$). PRISM adapter fine-tuned on 10K samples. Hyperparameters: $K=5$ (soft-masking), $G=8$ (GRPO group size), $\alpha=1.0$, $\beta=0.1$, $\epsilon=0.2$, $\gamma=0.5$ (composed prediction). Primary experiment: sweep $\tau_r \in \{0.001, 0.01, 0.05, 0.1, 0.2, 0.5\}$. All evaluations integrated into dInfer and SGLang.

Datasets. GSM8K [19] and MATH [20] for mathematical reasoning; HumanEval [21] for code generation. Policy training uses 50K prompts from OpenMathInstruct-2.

Metrics. Accuracy (exact match / pass@1), throughput (tokens/sec on A100), NFES, cache hit rate $\bar{\alpha} = \frac{1}{TL} \sum_{t,k} \alpha_t^k$, effective speedup vs. soft-routed-only baseline, refresh ratio $\frac{1}{T} \sum_t |\mathcal{Q}_t|/L$, and boundary depth $\frac{1}{T} \sum_t \ell_t^*$.

Ablations. *Architecture ablations:* (1) Remove the auxiliary (single soft-routed model only); (2) remove the steering policy (speculative caching with heuristic unmask/remask); (3) disable

composed prediction ($\gamma = 0$); (4) remove the agreement signal α_t^k from the policy input; (5) remove remark actions; (6) vary policy capacity ($N_\pi \in \{1, 2, 4\}$); (7) full τ_r sweep.

Steering-policy and caching ablations: The default system uses the composed prediction mechanism to drive all three action heads. We ablate enhancements to the caching subsystem that introduce access-pattern-aware positional speculation—predicting which positions will be acted on soon and speculatively refreshing their KV states, rather than refreshing only modified positions.

We ablate: (1) Refresh budget $B \in \{L/8, L/4, L/2, L\}$ (with $B=L$ recovering refresh-all); (2) candidate selection (learned $\mathcal{C}_t$ vs. sliding window vs. confidence top- B); (3) history-conditioned access features—augmenting the state with a per-position age counter a_t^k (steps since last action, clipped) and a learned embedding of the last action type (unmask/remask/none) to handle non-monotone revisits caused by remarking; (4) layer-wise refresh via an optional boundary layer (5) hybridization (AOAE unmask/remask + Elastic-trigger refresh vs. fully learned refresh).

Speculation-mode ablations: A central question is whether the auxiliary should predict both *where to unmask* and *which token to place*, or only the unmasking location while deferring token selection to a training-free method. We compare:

- **Full speculation (ours):** the auxiliary predicts both unmasking locations and draft tokens; the primary verifies both, and composed prediction biases toward the draft.
- **Location-only policy + training-free token selection:** the policy predicts only *where* to unmask/remask/cache, while the actual token at each unmasked position is selected by a training-free decoder (e.g., confidence thresholding as in Fast-dLLM [8], or the uniform baseline). This ablation isolates the value of the auxiliary’s token-level draft predictions and composed prediction from the value of learned positional steering alone.
- **Location-only policy + primary-only tokens:** the policy steers positions, but tokens are always sampled from the primary’s distribution p_t^k without composition ($\gamma = 0$) and without auxiliary drafts. This tests whether the auxiliary’s contribution is primarily in KV pre-caching or also in token prediction quality.

Auxiliary attention-mechanism ablations: The hard-routed auxiliary uses the same bidirectional full attention as the primary, which is the most expensive component of each forward pass. We ablate alternate attention mechanisms for the auxiliary to explore whether a cheaper attention pattern can maintain sufficient draft quality for high cache hit rates:

- **Full bidirectional attention (default):** standard attention over all positions.
- **Sliding-window attention:** each token attends only to a local window of w positions, reducing the auxiliary’s per-step cost from $O(L^2)$ to $O(Lw)$. Since diffusion models exhibit locality in their denoising dynamics (nearby tokens are often correlated), a sufficiently large window may preserve draft quality.
- **Sparse/strided attention:** every token attends to local neighbors plus a fixed set of strided global positions, balancing local context with long-range dependencies at reduced cost.
- **Linear attention:** replace softmax attention with a kernel-based linear approximation, reducing cost to $O(L)$ per layer. This is the most aggressive reduction and may degrade draft quality, but the primary’s full attention serves as a safety net.

References

- [1] J. Austin, D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

- [2] S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marber, S. Kuleshov, C. Jones, and V. Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.
- [3] J. Shi, K. Han, Z. Wang, A. Doucet, and M. Titsias. Simplified and generalized masked diffusion for discrete data. In *NeurIPS*, 2024.
- [4] A. Lou, C. Meng, and S. Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *ICML*, 2024.
- [5] J. Ou, Y. Song, and P. Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *arXiv:2406.03736*, 2024.
- [6] S. Nie, F. Zhu, Z. You, X. Zhang, J. Feng, Z. Zhang, C. Chen, G. Xu, Y. Dong, B. Liu, et al. Large language diffusion models. *arXiv:2502.09992*, 2025.
- [7] S. Nie, F. Zhu, C. Chen, et al. LLaDA 2.1: Speeding up text diffusion via token editing. *arXiv:2602.08676*, 2025.
- [8] R. Wu, Y. Qiu, D. Zheng, Z. Yu, L. Jiang, and L. Jiao. Fast-dLLM: Training-free fast generation for diffusion large language models. *arXiv preprint*, 2025.
- [9] M. Jazbec, T. X. Olausson, E. Hu, P. Hartout, D. Yogatama, E. Mathieu, and M. Cuturi. Learning unmasking policies for diffusion language models. *arXiv:2512.09106*, 2025.
- [10] J. Kim, K. Shah, V. Kontonis, S. Kakade, and S. Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *ICML*, 2025.
- [11] J. Kim, J. Li, B. Chen, and T. Hashimoto. PRISM: Plug-in remasking for inference-time self-correction of masked diffusions. *arXiv:2510.01384*, 2025.
- [12] Y. Ma, C. Li, Y. Liu, and G. Neubig. dKV-Cache: The cache for diffusion language models. In *NeurIPS*, 2025.
- [13] Q. Nguyen-Tri, M. Ranjan, and Z. Shen. Attention Is All You Need for KV Cache in Diffusion LLMs. In *ICLR*, 2026.
- [14] M. Hersche, S. Moor-Smith, T. Hofmann, and A. Rahimi. Soft-masked diffusion language models. *arXiv:2510.17206*, 2025.
- [15] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.
- [16] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.
- [17] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.
- [18] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.
- [19] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

- [20] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.
- [21] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [22] O. Kwon, Y. Kim, D. Kim, M. Kim, Y. Park, and J. W. Lee. MAGE: All-[MASK] Block Already Knows Where to Look in Diffusion LLMs. *arXiv preprint arXiv:2602.14209*, 2026.
- [23] Y. Ma, L. Du, L. Wei, K. Chen, Q. Xu, K. Wang, et al. dInfer: An Efficient Inference Framework for Diffusion Language Models. *arXiv:2510.08666*, 2025.